



Universidad Nacional Experimental de Guayana.

Vicerrectorado Académico.

Coordinación General de Pregrado.

Proyecto de Carrera: Ingeniería en Informática.

Unidad Curricular: Lenguajes y Compiladores.

Semestre 7 – Sección: 1 y 2.

Fases del Compilador y sus Tendencias.

Los ASTronautas

“Explorando el Universo de los Lenguajes, un Nodo a la vez”

Profesor:

Ing. Félix Márquez

Estudiantes:

C.I.: V-25.933.680. Alburquerque Sheen

C.I.: V-30.907.427. Antoima Mariangel

C.I.: V-28.475.271. García Carlos

C.I.: V-24.848.424. Varguillas Génesis.

Ciudad Guayana, Mayo del 2026.

Índice.

Contenido	pág.
Introducción	4
1. Fundamentos de la Compilación.....	5
1.1. ¿Qué es un compilador?	5
1.2. Diferencia clave: Compilador vs. Intérprete	5
1.3. Objetivos fundamentales de un compilador.....	6
2. Evolución Histórica e Hitos	6
2.1. Los inicios: cuando programar era (muy) complicado	6
2.2. La pionera: Grace Hopper y el primer compilador	7
2.3. El parteaguas: FORTRAN (1957)	7
2.4. El siguiente paso: autoalojamiento y teoría	7
2.5. La era moderna: GCC y el auge del software libre.....	8
3. Anatomía del Compilador: Las Fases	8
3.1 Etapa de Análisis (Front-End)	8
3.1.1 Análisis Léxico (Scanner).....	9
3.1.2 Análisis Sintáctico (Parser) y la importancia del AST	10
3.1.3 Análisis Semántico.....	12
3.2 Etapa de Síntesis (Back-End).....	15
3.2.1 Generación de Código Intermedio (IR)	15
3.2.2 Optimización de Código	17
3.2.3 Generación de Código Objeto (Código Máquina / Ensamblador).....	20
3.3 Componentes Transversales: Tabla de Símbolos y Manejo de Errores.....	22
3.3.1 Tabla de Símbolos.....	22
3.3.2 Manejo de Errores.....	25
4. Tendencias Modernas en la Construcción de Compiladores	27
4.1. Compilación Just-In-Time (JIT) y Máquinas Virtuales.....	28
4.2. Arquitecturas Modulares: LLVM y MLIR	29
4.3. Compilación Asistida por IA y Aprendizaje por Refuerzo (RL)	30
4.4. WebAssembly (WASM) y Seguridad en la Nube	31
4.5. Otras Tendencias Emergentes.....	32

5. Importancia en la Ingeniería en Informática actual	33
5.1. Te convierte en un mejor programador (aunque no construyas un compilador)	33
5.2. Las técnicas de compiladores están en todas partes.....	34
5.3. Es la base para diseñar lenguajes y herramientas propias.....	34
5.4. Es clave en seguridad informática (ciberseguridad)	35
5.5. Abre puertas a áreas especializadas y emergentes	36
5.6. Desarrolla pensamiento abstracto y resolución de problemas	36
Conclusión.	38
Referencias.....	40

Introducción.

Cuando escribimos un programa en un lenguaje de alto nivel, como Python, C++ o Java, la computadora no entiende directamente esas instrucciones. Los procesadores solo reconocen código máquina, es decir, secuencias de unos y ceros. Entonces, ¿cómo es posible que podamos programar con palabras y símbolos que nos resultan familiares? La respuesta está en los compiladores.

Un compilador es, básicamente, un traductor. Su trabajo consiste en tomar un programa escrito en un lenguaje fuente (el que usamos los programadores) y convertirlo en un programa equivalente en un lenguaje objeto (generalmente código máquina o algún tipo de representación de bajo nivel). Pero ojo, no es una simple traducción palabra por palabra; el compilador debe entender la estructura del programa, verificar que las reglas del lenguaje se cumplan y, de paso, intentar mejorar el rendimiento del código generado. En pocas palabras, es un programa que se especializa en... procesar otros programas.

Este tema es fundamental en la carrera de Ingeniería en Informática por varias razones. Primero, porque detrás de todo lenguaje de programación que usamos a diario hay uno o varios compiladores (o intérpretes) que hacen posible su ejecución. Segundo, porque muchas de las técnicas que se utilizan en la construcción de compiladores (como el análisis léxico, el análisis sintáctico o la optimización de código) también aparecen en otras áreas: desde la validación de archivos de configuración hasta el diseño de lenguajes de dominio específico (DSL). Y tercero, porque entender cómo funciona un compilador te cambia la forma de programar: empiezas a pensar en cómo se va a procesar tu código, qué errores puede detectar el compilador y cómo escribir instrucciones más claras y eficientes.

Vamos a recorrer cada una de las etapas que conforman un compilador clásico, desde que lee el código fuente hasta que genera el programa ejecutable. También dedicaremos un apartado a las tendencias modernas, porque los compiladores de hoy no son los mismos que los de hace veinte años: han aparecido la compilación Just-In-Time (JIT), arquitecturas modulares como LLVM, y hasta se está empezando a usar inteligencia artificial para optimizar código.

El informe está organizado en ocho secciones. Después de esta introducción, presentaremos los fundamentos básicos: qué es un compilador y en qué se diferencia de un intérprete. Luego haremos un breve recorrido histórico, para después entrar de lleno en la anatomía del compilador, desglosando las fases de análisis (front-end) y las fases de síntesis (back-end). Más adelante hablaremos de las tendencias actuales, cerraremos con la importancia de estos conceptos para un ingeniero en informática y, finalmente, expondremos las conclusiones del grupo.

1. Fundamentos de la Compilación

Antes de meternos de lleno en las fases internas de un compilador, conviene aclarar bien qué es exactamente esta herramienta y en qué se diferencia de otras, sobre todo del intérprete, que suele generar confusiones al principio de la carrera.

1.1. ¿Qué es un compilador?

En esencia, un compilador es un programa traductor. Su tarea principal es leer un programa escrito en un lenguaje fuente (generalmente de alto nivel, como C o Java) y convertirlo en un programa equivalente en un lenguaje objeto (normalmente código máquina o un lenguaje de bajo nivel) (Aho et al., 2007). Básicamente, le decimos a la máquina qué hacer usando palabras clave que entendemos nosotros, y el compilador se encarga de pasar esa receta a unos y ceros.

Sin embargo, no hay que confundir la traducción con la mera transformación de texto. Esta traducción implica un profundo análisis estructural. El compilador debe validar que nuestro código siga las reglas gramaticales del lenguaje (que no falten puntos y comas, que los paréntesis estén balanceados, etc.) y también que las operaciones tengan sentido (por ejemplo, que no intentemos sumar un texto con un número).

1.2. Diferencia clave: Compilador vs. Intérprete

Esta es una de las preguntas clásicas en la materia. Si bien ambos procesan código fuente, el momento en que actúan es distinto:

- ✓ **Compilador:** Lee todo el código fuente de una sola vez, lo traduce por completo a código máquina y guarda ese resultado (ejecutable). Una vez que tenemos el ejecutable, podemos correrlo sin necesidad de volver a tener el código fuente original ni el compilador (Louden, 2004).
- ✓ **Intérprete:** No genera un ejecutable independiente. Va leyendo el código fuente línea por línea (o instrucción por instrucción) y la va ejecutando directamente en el momento (Louden, 2004).

Analogía práctica: Piensa en un manual de instrucciones en inglés que necesitas traducir al español.

- ✓ El **compilador** sería un traductor profesional que toma todo el manual y te entrega la versión completa en español (el ejecutable). Una vez que tienes el manual traducido, puedes leerlo solo.

- ✓ El **intérprete** sería un amigo bilingüe que va leyendo una frase en inglés y te la traduce al español sobre la marcha mientras tú la ejecutas. Si quieres repetirla, tienes que traducirla de nuevo porque no guardó la versión final.

1.3. Objetivos fundamentales de un compilador

No basta con que el compilador traduzca bien. En la práctica profesional, un buen compilador debe cumplir dos objetivos clave:

- ✓ **Corrección:** El programa traducido debe comportarse exactamente igual que lo especificado en el código fuente. Sin esto, nada más sirve (Aho et al., 2007).
- ✓ **Eficiencia:** Idealmente, debe aprovechar el código para que corra rápido y ocupe menos memoria. Por eso las versiones modernas de los compiladores tienen modos de "Optimización" (Louden, 2004).

2. Evolución Histórica e Hitos

Para entender bien la arquitectura de un compilador moderno, conviene dar un vistazo rápido a cómo surgió todo esto. No se trata de memorizar fechas, sino de ver cómo pasamos de programar con cables y tarjetas perforadas a tener herramientas tan sofisticadas como LLVM o el compilador de Rust.

2.1. Los inicios: cuando programar era (muy) complicado

En la década de 1940 y principios de los 50, los programadores trabajaban directamente en código máquina, es decir, escribiendo secuencias de unos y ceros que el procesador entendía directamente. Como te podrás imaginar, esto era increíblemente tedioso, propenso a errores y nada productivo.

Para aliviar esto, surgieron los ensambladores, que no son más que traductores que convierten instrucciones nemotécnicas (como MOV, ADD, JMP) en el código binario que la máquina necesita. Si bien el ensamblador fue un gran avance, seguía siendo un lenguaje de muy bajo nivel, muy ligado a la arquitectura del procesador.

2.2. La pionera: Grace Hopper y el primer compilador

Aquí es donde entra en escena una figura clave: Grace Hopper. Esta matemática y militar estadounidense estaba convencida de que se podía programar usando palabras en inglés, haciendo la tecnología más accesible (Grace Hopper, 1952).

En 1952, Hopper y su equipo desarrollaron el A-0, considerado el primer compilador de la historia. El A-0 traducía instrucciones en inglés a código máquina, lo que supuso un cambio de paradigma total. Fue ella quien acuñó el término "compilar" (reunir, recopilar en inglés) para describir este proceso.

2.3. El parteaguas: FORTRAN (1957)

Si bien el A-0 fue el primero, el compilador que realmente demostró el potencial de esta tecnología fue el de FORTRAN (FORmula TRANslator), creado por un equipo de IBM liderado por John Backus y lanzado en 1957.

FORTRAN fue un hito por varias razones:

- ✓ **Primer lenguaje de alto nivel real:** Permitió a científicos e ingenieros escribir fórmulas matemáticas complejas de manera casi natural.
- ✓ **Eficiencia:** El equipo de Backus logró que el código generado por el compilador fuera casi tan eficiente como el escrito a mano por un programador experto en ensamblador. Esto fue crucial para que la industria y la ciencia confiaran en los compiladores.
- ✓ **Éxito comercial:** FORTRAN se convirtió en el lenguaje estándar para la computación científica y demostró que este nuevo modelo de programación era viable y rentable.

El desarrollo de este primer compilador de FORTRAN fue un proyecto titánico, que se estima requirió **18 años-persona** de trabajo.

2.4. El siguiente paso: autoalojamiento y teoría

Poco después, en 1962, en el MIT, Tim Hart y Mike Levin crearon el primer compilador autoalojado (o *self-hosting*) para el lenguaje Lisp. ¿Qué significa esto? Que escribieron un compilador en el propio lenguaje Lisp. Es como si un traductor de inglés a español fuera capaz de traducirse a sí mismo a otro idioma. Esto se conoce

como bootstrapping y es el método que se usa hoy para construir la mayoría de los lenguajes de programación.

En paralelo, durante los 60 y 70, personajes como Noam Chomsky (con su teoría de gramáticas formales) y Donald Knuth (con sus trabajos sobre análisis sintáctico LR) pusieron las bases teóricas y matemáticas de los compiladores, transformando lo que era un arte casi de artesanía en una rigurosa disciplina de la ingeniería.

2.5. La era moderna: GCC y el auge del software libre

Durante mucho tiempo, los compiladores eran herramientas comerciales y altamente costosas. La situación cambió radicalmente con la llegada del GCC (GNU Compiler Collection), iniciado por Richard Stallman en 1987 como parte del proyecto GNU.

GCC fue el primer compilador libre, gratuito y de alta calidad. Su impacto fue enorme:

- ✓ **Democratizó el acceso:** Cualquier persona, universidad o empresa podía usar un compilador profesional sin pagar licencias.
- ✓ **Hizo posible Linux:** El sistema operativo Linux necesita un compilador para poder crearse a sí mismo, y GCC fue la herramienta que lo hizo posible.

3. Anatomía del Compilador: Las Fases

Ahora que ya sabemos qué es un compilador y su evolución histórica, toca entrar en lo que realmente nos interesa: cómo funciona por dentro. Vamos a desglosar cada una de sus fases, desde que recibe el código fuente como texto plano hasta que genera un ejecutable listo para correr.

3.1 Etapa de Análisis (Front-End)

El Front-End es la primera mitad del compilador. Su trabajo principal es entender el código fuente: leerlo, verificar que esté bien escrito y que tenga sentido dentro de las reglas del lenguaje. Como te imaginarás, no es trivial. El Front-End se divide en tres fases que trabajan en cadena, como una línea de ensamblaje.

3.1.1 Análisis Léxico (Scanner)

¿Qué es?

El análisis léxico es la primera fase de cualquier compilador. Suena más complejo de lo que realmente es: básicamente, se encarga de leer el código fuente carácter por carácter y agruparlos en unidades con significado llamadas tokens . Un token es como una "palabra" para el compilador: un if, un while, una variable llamada contador, un número 123, un operador +, etc.

¿Cómo funciona?

El analizador léxico (también llamado *lexer* o *scanner*) recorre el texto de entrada y aplica un conjunto de **expresiones regulares** que definen cómo se ven los tokens válidos. Por ejemplo, una expresión regular para reconocer números enteros podría ser **[0-9]+** (uno o más dígitos). Para reconocer identificadores (nombres de variables), algo como **[a-zA-Z_][a-zA-Z0-9_]*** (una letra o guión bajo, seguido de letras, números o guiones bajos).

El lexer no solo identifica tokens, sino que también:

- ✓ **Elimina espacios en blanco, tabulaciones y comentarios** (porque al compilador no le interesan para entender la lógica del programa)
- ✓ **Cuenta líneas y columnas** para poder reportar errores con precisión ("error en la línea 10, columna 5")
- ✓ **Asocia un tipo y un valor** a cada token. Por ejemplo, el token NUMERO puede tener el valor 42.

Herramientas para construir analizadores léxicos

En la práctica real, nadie escribe un analizador léxico a mano desde cero (aunque técnicamente se puede). Se usan generadores de analizadores léxicos, que son programas que toman una descripción de los tokens (en forma de expresiones regulares) y generan automáticamente el código fuente del lexer.

La herramienta clásica en el mundo Unix/Linux es Flex (Fast Lexical Analyzer). También existen Lex (la original), JLex para Java, y versiones integradas en frameworks más grandes como ANTLR.

Un programa en Flex tiene tres secciones:

- ✓ **Definiciones:** Aquí se definen macros y expresiones regulares reutilizables (ej: **DIGITO [0-9]**)
- ✓ **Reglas:** La parte central, donde se asocia cada patrón regular con una acción en C a ejecutar cuando se reconoce
- ✓ **Código de usuario:** Funciones auxiliares en C

Ejemplo práctico:

Imagina el siguiente código fuente: **x = 3 + 5**

El analizador léxico produciría algo como:

Token IDENTIFICADOR (valor: "x")

Token ASIGNACION (valor: "=")

Token NUMERO (valor: 3)

Token SUMA (valor: "+")

Token NUMERO (valor: 5)

¿Y si algo sale mal?

Si el lexer encuentra un carácter que no coincide con ninguna expresión regular definida, se genera un error léxico. Por ejemplo, si tu lenguaje no permite el símbolo **@** y aparece en el código, el lexer reporta "carácter no válido".

3.1.2 Análisis Sintáctico (Parser) y la importancia del AST

¿Qué es?

El análisis sintáctico es la segunda fase del compilador. Si el análisis léxico se pregunta "¿qué palabras hay?", el análisis sintáctico se pregunta "¿están esas palabras en el orden correcto según las reglas del lenguaje?".

Mientras el lexer trabaja con caracteres y tokens, el parser trabaja con la secuencia de tokens generada por el lexer. Su trabajo es verificar que esa secuencia cumpla con **la gramática del lenguaje**.

Gramáticas libres de contexto (GLC)

Para definir la sintaxis de un lenguaje de programación se usan las gramáticas libres de contexto. Una GLC consta de:

- ✓ **Símbolos terminales:** son los tokens (ej: if, (, }, variable, 123)
- ✓ **Símbolos no terminales:** son categorías gramaticales (ej: sentencia, expresion, declaracion)
- ✓ **Producciones:** reglas que dicen cómo se construye un no terminal a partir de otros elementos (ej: $\text{sentencia} ::= \text{if '(' expresion ')'} \text{sentencia}$)
- ✓ **Símbolo inicial:** el no terminal del que parte toda la gramática (normalmente programa)

Un ejemplo muy simple de producción sería:

```
expresion ::= expresion '+' expresion  
| expresion '-' expresion  
| NUMERO  
| IDENTIFICADOR
```

Esto dice que una expresión puede ser: una suma, una resta, un número o una variable.

El árbol sintáctico y el AST

Cuando el parser valida una secuencia de tokens, puede construir una representación gráfica llamada árbol sintáctico (parse tree). Este árbol muestra cómo los tokens se agrupan según las producciones de la gramática.

Pero ese árbol suele tener mucha información redundante. Por eso la mayoría de los compiladores construyen en su lugar un AST (Abstract Syntax Tree). El AST es una versión simplificada y más compacta del árbol sintáctico. Omite detalles superfluos como paréntesis, punto y coma si no son necesarios, y algunos nodos intermedios.

Y justo ahí está nuestra señal de identidad: el AST es la estructura que convierte una expresión como **a + b * c** en un árbol donde se ve claramente que la multiplicación tiene prioridad sobre la suma. Como grupo "ASTronautas", navegamos por esa estructura para entender cómo el compilador "piensa" tu código.

Tipos de parsers

Existen dos familias principales de analizadores sintácticos:

Tipo	Enfoque	Ejemplos
Descendentes (LL)	Parten del símbolo inicial y van derivando hasta llegar a los tokens. Son más fáciles de escribir a mano.	LL(1), parser recursivo descendente
Ascendentes (LR)	Parten de los tokens y van agrupándolos hasta llegar al símbolo inicial. Son más potentes pero más complejos.	LR(0), SLR, LALR, LR(1)

Herramientas como **Bison** (para C) o **JavaCUP** (para Java) permiten generar parsers a partir de una descripción de la gramática.

Manejo de errores sintácticos

Si el parser encuentra una secuencia de tokens que no se ajusta a ninguna producción de la gramática, se produce un error sintáctico. Por ejemplo: un `else` sin un `if` que lo preceda, o un paréntesis que se abre pero no se cierra. Un buen parser no solo detecta el error, sino que intenta recuperarse para seguir analizando el resto del código y reportar múltiples errores en una sola pasada.

3.1.3 Análisis Semántico

¿Qué es?

El análisis semántico es la tercera fase del compilador (Hernandez, s.f.). Mientras que las fases anteriores verificaban la forma (léxico) y la estructura (sintaxis), esta fase verifica el significado y la coherencia lógica del programa. O dicho de forma más simple: el parser sabe que `x = y + z` está bien escrito, pero el análisis semántico es el que comprueba que `x`, `y` y `z` sean variables declaradas, que tengan tipos compatibles para sumarse, y que la asignación tenga sentido.

¿Qué comprueba el análisis semántico?

- ✓ **Uso de identificadores:** que las variables se declaren antes de usarse, que no se declaren dos veces en el mismo ámbito.
- ✓ **Coherencia de tipos:** que no intentes sumar un string con un entero, o pasar un argumento de tipo incorrecto a una función.
- ✓ **Ámbitos:** que no accedas a una variable fuera de su bloque de declaración (ej: una variable local dentro de una función no puede ser usada fuera de ella).
- ✓ **Llamadas a funciones:** que el número y tipo de argumentos coincida con la definición de la función.

La tabla de símbolos

Para realizar todas estas comprobaciones, el análisis semántico necesita una estructura de datos clave: la tabla de símbolos (LinkedIn Learning, s.f.). Piensa en ella como un diccionario que el compilador va llenando a medida que analiza el código. Por cada identificador (variable, función, clase, constante, etc.), la tabla guarda información como:

Atributo	¿Qué guarda?	Ejemplo
Nombre	El identificador en sí	contador
Tipo	El tipo de dato	entero, cadena, booleano
Ámbito	Dónde es válido	global, local, argumento
Valor (si es constante)	Valor inicial	10
Ubicación	Dirección de memoria o registro	(lo define el backend)

La tabla de símbolos se crea durante el análisis léxico y sintáctico (cuando se detectan tokens que son identificadores) y se consulta intensivamente durante el análisis semántico y la generación de código.

Tipado estático vs dinámico

El análisis semántico está muy ligado al **sistema de tipos** del lenguaje:

- ✓ **Tipado estático** (C, Java, Rust): los tipos de las variables se conocen en tiempo de compilación. El compilador puede detectar errores de tipo antes de que el programa se ejecute.
- ✓ **Tipado dinámico** (Python, JavaScript, PHP): los tipos se verifican en tiempo de ejecución. El compilador (o intérprete) delega esa responsabilidad.

Los lenguajes estáticos realizan un análisis semántico mucho más riguroso. De hecho, buena parte de la "fama" de que Java o C sean "pesados" viene de los estrictos que son en esta fase: no permiten ambigüedades ni operaciones raras.

¿Qué pasa si hay errores semánticos?

Si el análisis semántico encuentra una incoherencia (ej: sumar un texto con un número), genera un error semántico (Hernandez, s.f.). A diferencia de los errores léxicos o sintácticos (que dicen que algo está mal escrito), los errores semánticos dicen que no se puede hacer lo que pides, aunque esté bien escrito. Por ejemplo:

- ✓ "No se puede dividir un string por un entero"
- ✓ "Función 'calcular' no definida"
- ✓ "Variable 'total' no declarada"

Relación con el AST

El análisis semántico recorre el AST generado por el parser. A medida que visita cada nodo, realiza las comprobaciones correspondientes y va anotando el árbol con información adicional (tipo de cada expresión, referencia a la entrada en la tabla de símbolos, etc.). Este AST "anotado" se le pasa luego al backend para la generación de código.

Ejemplo práctico

Código fuente:

```
int a = 5;  
int b = "hola";  
int c = a + b;
```

El análisis semántico detectaría:

- ✓ **a** declarada correctamente como entero.
- ✓ **b** declarada como entero pero se le asigna un string → **error semántico**.
- ✓ **c** intenta sumar entero (**a**) con string (por el error anterior, también se reporta inconsistencia).

3.2 Etapa de Síntesis (Back-End)

Una vez que el Front-End ha terminado su trabajo, tenemos un AST anotado (con información de tipos, tabla de símbolos, etc.). Ahora toca convertir esa representación abstracta en código que realmente pueda ejecutar una máquina. Ese es el trabajo del Back-End.

El Back-End se compone de tres fases principales que trabajan en cadena: Generación de Código Intermedio, Optimización y Generación de Código Final.

3.2.1 Generación de Código Intermedio (IR)

¿Qué es?

La generación de código intermedio es la primera fase del Back-End (Aho et al., 2007). Su trabajo es traducir el AST (que todavía es una representación de alto nivel) a un lenguaje intermedio (IR, por sus siglas en inglés). El IR es como un "punto medio" entre el lenguaje fuente y el código máquina: es más sencillo que el original, pero todavía no es instrucciones de procesador.

¿Para qué sirve un IR?

Tener una representación intermedia tiene varias ventajas:

- ✓ **Independencia de arquitectura:** El Front-End solo necesita generar IR. Luego, el Back-End traduce ese IR a x86, ARM, RISC-V, etc. Así se reutiliza el trabajo.
- ✓ **Optimización más fácil:** Es más sencillo optimizar un IR bien diseñado que optimizar directamente código máquina.
- ✓ **Portabilidad:** El mismo IR puede servir para múltiples lenguajes fuente. Por ejemplo, LLVM IR es usado por C, Rust, Swift, Kotlin, etc.

Tipos de representación intermedia

Existen varios estilos de IR (Louden, 2004):

Tipo	Característica	Ejemplo
Código de tres direcciones	Instrucciones del tipo $x = y \text{ op } z$. Es el más común en compiladores académicos.	$t1 = a + b$ $t2 = t1 * c$
Forma estática de asignación única (SSA)	Cada variable se asigna una sola vez. Facilita muchas optimizaciones.	Usado por LLVM IR
Notación postfija	Similar a la notación polaca inversa.	$a \ b \ + \ c \ *$
Árboles de sintaxis abstracta (AST)	El mismo AST, pero a veces se usa como IR si es sencillo.	El que ya conocemos

Ejemplos de IR modernos

IR	¿Dónde se usa?	Característica
LLVM IR	LLVM (C, Rust, Swift, Zig, etc.)	SSA, tipado fuerte, optimizaciones muy potentes
WASM (WebAssembly) Text Format	WebAssembly (navegadores, edge computing)	Seguro, portable, pensado para la web
Java Bytecode	JVM (Java, Kotlin, Scala, Groovy)	Orientado a máquina virtual, no a hardware real
MLIR	Compiladores de IA y dominios específicos	Multi-nivel, extensible

Un ejemplo sencillo (código de tres direcciones)

Si tenemos la expresión en código fuente:

$$x = (a + b) * c$$

El Front-End genera un AST, y luego el generador de IR produce algo como:

$$t1 = a + b$$
$$t2 = t1 * c$$
$$x = t2$$

Cada instrucción es muy simple, y se introducen variables temporales (t1, t2) para almacenar resultados intermedios.

¿Por qué es útil esto?

Porque más adelante, el optimizador puede:

- ✓ Ver que t1 solo se usa una vez (para t2)
- ✓ Si **a**, **b** y **c** son constantes, calcular todo en tiempo de compilación
- ✓ Reordenar instrucciones para mejorar el rendimiento

3.2.2 Optimización de Código

¿Qué es?

La optimización de código es la fase que **mejora el IR** sin cambiar su comportamiento (Aho et al., 2007). El objetivo es que el programa final sea más rápido, ocupe menos memoria, o consuma menos energía (dependiendo de lo que priorice el compilador). Lo mejor de todo es que estas mejoras son automáticas: el programador no tiene que hacer nada especial (más allá de activar la optimización con banderas como **-O2** en GCC).

¿Es obligatorio optimizar?

No. Un compilador puede funcionar perfectamente sin optimizar. Pero el código generado será más lento y más grande. En la práctica, todos los compiladores modernos tienen al menos un nivel básico de optimización.

Clasificación de las optimizaciones

Tipo	¿Qué hace?	Ejemplo
Optimizaciones independientes de la máquina	Se aplican sobre el IR, sin importar el procesador final	Eliminación de código muerto, propagación de constantes
Optimizaciones dependientes de la máquina	Aprovechan características específicas del procesador	Uso de instrucciones SIMD, organización de registros

Técnicas comunes de optimización

A continuación, algunas de las más importantes:

✓ Eliminación de código muerto (Dead Code Elimination)

Si hay código que nunca se ejecuta o cuyos resultados nunca se usan, se puede eliminar.

// Código original

```
int x = 5;
int y = 10;
x = 20; // La primera asignación a x se pierde
return x;
```

// Optimizado

```
int x = 20;
return x;
```

✓ Propagación de constantes (Constant Propagation)

Si una variable siempre tiene el mismo valor conocido, se reemplaza ese valor directamente.

// Código original

```
int a = 5;
```

```
int b = a + 3;
```

```
// Optimizado
```

```
int a = 5;
```

```
int b = 8;
```

✓ **Desenrollado de bucles (Loop Unrolling)**

Se repite el cuerpo del bucle varias veces para reducir la cantidad de saltos y comprobaciones.

```
// Código original
```

```
for (int i = 0; i < 4; i++) {  
    suma += arr[i];  
}
```

```
// Optimizado (desenrollado)
```

```
suma += arr[0];  
suma += arr[1];  
suma += arr[2];  
suma += arr[3];
```

✓ **Eliminación de subexpresiones comunes (CSE)**

Si la misma expresión se calcula varias veces, se calcula una sola vez y se reutiliza.

```
// Código original
```

```
int a = (x + y) * z;  
int b = (x + y) * w;
```

```
// Optimizado
```

```
int temp = x + y;  
int a = temp * z;  
int b = temp * w;
```

¿Hay riesgos en la optimización?

Sí, aunque pocos. A veces una optimización demasiado agresiva puede:

- ✓ **Cambiar el comportamiento** si el código original dependía de efectos secundarios raros.
- ✓ **Aumentar el tamaño del ejecutable** (desenrollar bucles muy largos, por ejemplo).
- ✓ **Alargar el tiempo de compilación** significativamente.

Por eso los compiladores tienen **niveles** de optimización (**-O0, -O1, -O2, -O3, -Os**, etc.). El programador elige cuál usar según sus necesidades.

3.2.3 Generación de Código Objeto (Código Máquina / Ensamblador)

¿Qué es?

Esta es la última fase del compilador (Aho et al., 2007). Toma el IR (ya optimizado) y lo convierte en código máquina real que el procesador puede ejecutar. Dependiendo del compilador y de las opciones, puede generar:

- ✓ **Código ensamblador** (archivo **.s** o **.asm**), que luego un ensamblador convertirá a binario.
- ✓ **Código objeto** (archivo **.o** o **.obj**), que ya está en binario pero sin enlazar.
- ✓ **Ejecutable final** (**.exe, .out**), listo para correr.

¿Qué decisiones debe tomar el Back-End?

Generar código máquina no es automático ni directo. El Back-End debe resolver varios problemas:

- ✓ **Asignación de registros:** Las variables y temporales viven en memoria, pero el procesador tiene pocos registros (16-32 en x86-64). ¿Quién ocupa cada registro?
- ✓ **Selección de instrucciones:** Una misma operación se puede implementar con varias instrucciones diferentes. ¿Cuál elegir (rápida, pequeña, etc.)?
- ✓ **Ordenamiento de instrucciones:** Reordenar instrucciones para evitar que el procesador tenga que esperar (pipeline, cachés, etc.).

Arquitecturas objetivo

Un compilador moderno puede generar código para múltiples arquitecturas. Algunas de las más comunes:

Arquitectura	¿Dónde se usa?	Ejemplos de compiladores
x86_64	PCs, servidores (Intel/AMD)	GCC, Clang, MSVC
ARM64	Móviles, tablets, Macs modernas	GCC, Clang, LLVM
RISC-V	Hardware libre, embebidos, investigación	GCC, LLVM
WASM	Navegadores, edge computing (no es hardware real)	Emscripten, wasm64

Un ejemplo muy simplificado

Supongamos que el IR optimizado tiene:

t1 = a + b

En un procesador x86_64, el Back-End podría generar:

mov rax, [a] ; copia el valor de 'a' al registro rax

add rax, [b] ; suma el valor de 'b' a rax

mov [t1], rax ; guarda el resultado en t1

En ARM64 sería algo como:

ldr x0, [a] ; carga 'a' en registro x0

ldr x1, [b] ; carga 'b' en registro x1

add x0, x0, x1 ; suma x0 + x1, resultado en x0

str x0, [t1] ; guarda el resultado en t1

¿Y qué pasa con las variables temporales?

Las variables temporales (**t1**, **t2**, etc.) normalmente no existen en el ejecutable final. El Back-End asigna registros para ellas, o las "elimina" si puede integrarlas en otras operaciones. Ese es el trabajo del asignador de registros.

El rol del ensamblador y el enlazador

El compilador puro termina generando código objeto o ensamblador. Luego:

- ✓ **Ensamblador**: convierte el código ensamblador a binario (código objeto).
- ✓ **Enlazador (linker)**: combina varios archivos objeto, resuelve referencias entre ellos (ej: una función definida en otro archivo) y genera el ejecutable final.

En la práctica, muchos compiladores (como GCC o Clang) llaman automáticamente al ensamblador y al enlazador, así que el usuario solo ve un paso: **gcc programa.c -o programa**.

3.3 Componentes Transversales: Tabla de Símbolos y Manejo de Errores

Hay dos componentes que no pertenecen exclusivamente al Front-End ni al Back-End, sino que atraviesan todo el compilador. Están presentes desde las primeras fases hasta las últimas, y son fundamentales para que el compilador funcione correctamente.

3.3.1 Tabla de Símbolos

¿Qué es?

La tabla de símbolos es una estructura de datos que el compilador utiliza para almacenar información sobre los identificadores del programa (LinkedIn Learning, s.f.). Un identificador puede ser una variable, una constante, una función, una clase, un parámetro, un tipo definido por el usuario, etc.

Piénsala como una base de datos pequeña que el compilador va llenando a medida que lee el código. Cada vez que encuentra una declaración (ej: **int contador**;;), la anota en la tabla. Luego, cuando encuentra un uso (ej: **contador = contador + 1**;;), consulta la tabla para saber qué es **contador** y si está permitido usarlo ahí.

¿Qué información guarda la tabla de símbolos?

Atributo	¿Qué almacena?	Ejemplo
Nombre	El identificador en sí	contador, calcular_promedio
Tipo	El tipo de dato (si aplica)	int, float, char[], bool
Categoría	Qué tipo de símbolo es	variable, constante, función, parámetro, tipo
Ámbito	Dónde es válido (alcance)	global, local al bloque, argumento de función
Valor (si es constante conocida)	Valor en tiempo de compilación	10, "Hola"
Ubicación	Dónde está almacenado en memoria	dirección, registro, offset en la pila
Atributos adicionales	Depende del lenguaje	static, volatile, const, public

¿Por qué no puede ser una tabla simple?

Porque los lenguajes de programación tienen ámbitos anidados. Por ejemplo, en C puedes tener:

```
int x = 10;     // x global
```

```
void funcion() {
```

```
    int x = 20;   // x local (oculta a la global)
```

```
    printf("%d", x); // imprime 20, no 10
```

```
}
```

Aquí hay dos variables con el mismo nombre **x**, pero en ámbitos diferentes. La tabla de símbolos debe manejar esta situación. La solución típica es tener una estructura de tablas encadenadas (como una pila):

Nivel de la pila	Ámbito	Símbolos
Fondo	Global	x (10)
Tope	Función funcion	x (20) - oculta al global

Cuando el compilador busca **x**, empieza desde el tope (el ámbito más interno) y va bajando hasta encontrarlo.

¿Cuándo se usa la tabla de símbolos?

Fase del compilador	¿Cómo se usa la tabla de símbolos?
Análisis Léxico	Casi no se usa. Solo detecta identificadores como tokens, pero no los interpreta.
Análisis Sintáctico	Puede empezar a registrar declaraciones cuando las encuentra.
Análisis Semántico	Uso intensivo. Verifica que los identificadores existan, que los tipos coincidan, que no se declaren dos veces, etc.
Generación de IR / Back-End	Consulta la ubicación de cada símbolo (dirección, registro) para generar el código correcto.

Ejemplo práctico

Código fuente:

```
int main() {
    int a = 5;
    int b = a + 3;
    return 0;
}
```


La tabla de símbolos después del análisis contendría:

Nombre	Tipo	Categoría	Ámbito	Ubicación
main	int	función	global	entrada 0x1000
a	int	variable	main	offset -4 (pila)
b	int	variable	main	offset -8 (pila)

Cuando el Back-End genere código para **int b = a + 3**, consultará la tabla y sabrá que **a** está en **offset -4** de la pila, y que **b** debe guardarse en **offset -8**.

3.3.2 Manejo de Errores

¿Qué es?

El manejo de errores es el conjunto de estrategias que usa el compilador para **detectar, reportar y (si es posible) recuperarse de errores** en el código fuente (Aho et al., 2007). Un compilador no solo debe encontrar errores, sino también ayudarle al programador a entenderlos y corregirlos.

Tipos de errores que puede detectar un compilador

Tipo de error	¿Cuándo ocurre?	Ejemplo
Error léxico	El lexer encuentra un carácter no válido	int a = @5; (el @ no está definido)
Error sintáctico	La secuencia de tokens no sigue la gramática	if (x > 0 { (falta el paréntesis de cierre)
Error semántico	El significado es incorrecto aunque la sintaxis está bien	int a = "hola"; (no se puede asignar texto a un entero)

¿Qué debe hacer un compilador cuando encuentra un error?

Acción	Explicación	¿Es obligatorio?
Detectar	Reconocer que algo está mal	Sí
Reportar	Mostrar un mensaje claro (línea, columna, descripción)	Sí
Recuperarse	Seguir analizando para encontrar más errores	Deseable, no obligatorio
Corregir	Intentar arreglar el error automáticamente	No (excepto en editores o linters)

Estrategias de recuperación de errores

Si el compilador se detiene en el primer error, el programador solo verá un error por compilación y tendrá que corregir, compilar de nuevo, ver el siguiente, etc. Es muy ineficiente. Por eso los compiladores implementan mecanismos de recuperación:

Estrategia	¿Cómo funciona?	Ejemplo
Recuperación en modo pánico	Se descartan tokens hasta encontrar un punto seguro (ej: punto y coma, }, end)	Si falta un paréntesis, se salta hasta el siguiente ;
Recuperación con producción de error	La gramática del lenguaje incluye reglas especiales para errores comunes	sentencia_error ::= error ';' ;
Corrección global	Se analizan varias alternativas para minimizar cambios (más complejo)	Usado por algunos parsers LR

¿Cómo se ven los mensajes de error modernos?

Un compilador moderno no solo dice "Error en línea 10". Da información detallada:

programa.c:10:5: error: 'contador' undeclared (first use in this function)

```
10 |   contador = contador + 1;
```

| ~~~~~

programa.c:10:5: note: each undeclared identifier is reported only once for each function it appears in

Esto incluye:

- ✓ Archivo, línea y columna exacta
- ✓ Descripción del error
- ✓ Subrayado o indicador visual del token problemático
- ✓ Sugerencias o notas adicionales

Relación con los componentes del compilador

Fase	¿Qué errores detecta?	¿Qué hace con ellos?
Léxico	Caracteres inválidos, tokens mal formados	Reporta y continúa con el siguiente carácter
Sintáctico	Secuencia incorrecta de tokens	Intenta recuperarse (modo pánico) y sigue analizando
Semántico	Incoherencias lógicas (tipos, ámbitos, declaraciones)	Reporta y marca el AST para no generar código incorrecto

Consejo para el programador

Los mensajes de error del compilador pueden parecer abrumadores al principio, pero con práctica se aprenden a leer. La regla de oro: centrarse siempre en el primer error reportado. Los siguientes suelen ser consecuencia del primero.

4. Tendencias Modernas en la Construcción de Compiladores

Los compiladores han evolucionado muchísimo desde los días de FORTRAN y el primer GCC. Hoy en día, estamos viendo un cambio de paradigma: ya no se trata solo de traducir código, sino de adaptarse dinámicamente, optimizar con inteligencia artificial y

ejecutar código de forma segura en cualquier lugar. A continuación, las tendencias más importantes que están marcando el rumbo de la industria y la academia en 2026.

4.1. Compilación Just-In-Time (JIT) y Máquinas Virtuales

Tradicionalmente, un compilador traduce todo el código antes de ejecutarlo (AOT, *Ahead-Of-Time*). Pero eso no siempre es la mejor opción. La compilación JIT (Just-In-Time) mezcla lo mejor de los dos mundos: empieza ejecutando el código de forma interpretada o compilada rápidamente, pero va perfilando qué partes del código se ejecutan más (las "rutas calientes") y las recompila sobre la marcha con optimizaciones agresivas.

Ventajas del JIT:

- ✓ **Adaptabilidad:** Se optimiza según el uso real, no según lo que el programador cree que se usará
- ✓ **Menor tiempo de inicio:** Se optimiza según el uso real, no según lo que el programador cree que se usará
- ✓ **Optimización por perfil:** Si una función se llama mil veces, se recompila más optimizada; si se llama una vez, se deja rápida sin más

Motores JIT famosos que usas todos los días:

Motor	¿Dónde está?	Lenguajes que ejecuta
V8	Google Chrome, Node.js	JavaScript, TypeScript
HotSpot	Java (JVM)	Java, Kotlin, Scala, Groovy
PyPy	Python (alternativa a CPython)	Python
.NET CLR	.NET Framework / .NET Core	C#, F#, VB.NET

El caso específico de WebAssembly (WASM) y JIT

WebAssembly (WASM) es un formato de bytecode de bajo nivel diseñado para ser rápido, seguro y portable. Los navegadores lo ejecutan con un runtime JIT: el WASM se compila a código nativo justo antes de ejecutarse.

Lo interesante es que WASM ya no es solo "para navegadores". Con WASI (WebAssembly System Interface), puedes ejecutar binarios WASM fuera del navegador, en servidores, dispositivos IoT o edge computing, con un sandboxing muy seguro y acceso controlado a los recursos del sistema.

4.2. Arquitecturas Modulares: LLVM y MLIR

LLVM (Low Level Virtual Machine) es una infraestructura de compilación que tomó esa idea y la llevó a otro nivel. No es un compilador en sí, sino un conjunto de bibliotecas reutilizables para construir compiladores.

¿Por qué LLVM cambió el juego?

Antes	Con LLVM
Cada lenguaje necesitaba su propio compilador completo (front-end + back-end)	Cada lenguaje solo necesita su front-end. El back-end (generación de código para x86, ARM, etc.) es compartido
Escribir un compilador era un proyecto titánico	Escribir un compilador es más accesible (sigue siendo difícil, pero menos)

Lenguajes que usan LLVM: Rust, Swift, Kotlin Native, Zig, Crystal, y por supuesto C/C++ con Clang.

MLIR (Multi-Level Intermediate Representation)

MLIR es el "hermano pequeño" de LLVM, pero con una idea más ambiciosa: permitir múltiples niveles de abstracción dentro de un mismo compilador. Esto es especialmente útil para:

- ✓ **Compiladores de IA** (TensorFlow, PyTorch, JAX)
- ✓ **Lenguajes de dominio específico (DSL)**
- ✓ **Hardware especializado** (TPUs, NPU, GPUs)

La investigación más reciente (2026) muestra que MLIR se usa para optimizar modelos de deep learning generados desde PyTorch y JAX, logrando un rendimiento competitivo o superior a bibliotecas como Torch Inductor y XLA. También se utiliza para transpilar programas científicos (por ejemplo, modelos oceánicos en Fortran o Julia) para que corran en aceleradores de IA en la nube.

4.3. Compilación Asistida por IA y Aprendizaje por Refuerzo (RL)

Esta es probablemente la tendencia más revolucionaria y la que está cambiando todo. La idea es simple: ¿por qué no usar técnicas de inteligencia artificial para decidir cómo optimizar el código?

¿Qué tipo de IA se está usando?

Técnica	¿Para qué se usa?	Ejemplo concreto
Aprendizaje por refuerzo (RL)	Decidir qué optimizaciones aplicar (pases, desenrollado de bucles, etc.) elige la mejor secuencia para cada función	MLIRCompilerEnv: agentes RL que predicen pases de optimización en MLIR
Redes neuronales	Predecir qué caminos de ejecución son más probables (perfilado estático)	MLIR RL: optimización de bucles en código generado desde PyTorch
Modelos de lenguaje (LLMs)	Generar código, documentación, o incluso parches de optimización	LLM-VeriOpt: combinación de LLM con RL y verificación formal

Ejemplo real (2026): La conferencia CGO 2026 presentó **MLIR RL**, un entorno de aprendizaje por refuerzo específico para MLIR. Se entrenó un agente RL para optimizar

código Linalg (un dialecto de MLIR para operaciones de álgebra lineal) generado desde PyTorch. Los resultados demuestran que es posible aprender **políticas de optimización** que superan a las heurísticas escritas a mano.

Y todavía más allá: Se están diseñando sistemas **compilador-runtime co-diseñados** con IA. La idea es que el compilador emita **múltiples variantes del kernel** (diferentes formas de organizar bucles, memoria, etc.), y luego el runtime, usando modelos de ML ligeros (árboles de decisión o redes neuronales pequeñas), elige **en tiempo real** la mejor variante según el estado actual del sistema (carga de memoria, uso de CPU, latencia de red, etc.) .

4.4. WebAssembly (WASM) y Seguridad en la Nube

WebAssembly nació como un "formato de bytecode para navegadores", pero hoy es mucho más. Es una **máquina virtual segura y portable** que permite ejecutar código compilado (C, C++, Rust, Go, etc.) en cualquier entorno.

¿Qué hace especial a WASM?

Característica	Explicación
Sandboxing	El código WASM se ejecuta en un entorno aislado. No puede acceder a archivos, red o sistema operativo a menos que el host se lo permita explícitamente
Portabilidad	El mismo binario WASM corre en Windows, Linux, macOS, navegadores, servidores edge, dispositivos IoT...
Rendimiento	Casi nativo (no tan rápido como C optimizado, pero mucho más rápido que JavaScript)
Seguridad	Validación de bytecode antes de ejecutar. No hay punteros dangling, no hay buffer overflows

Casos de uso actuales (2026):

- ✓ **Ejecución de plugins en editores** (Figma, AutoCAD, VS Code)

- ✓ **Funciones serverless** (Cloudflare Workers, Fastly Compute@Edge)
- ✓ **Microservicios seguros** (junto con WASI, la interfaz de sistema)
- ✓ **Sandbox para agentes de IA** WasmEdge, un runtime WASM, se usa para ejecutar módulos personalizados generados por IA de forma segura. Por ejemplo, un pipeline de voz: la transcripción entra, pasa por un módulo WASM hecho con Claude Code (IA) que la procesa, y luego va al sintetizador de voz. Todo el código intermedio es injected by AI pero aislado por WASM.

4.5. Otras Tendencias Emergentes

Tendencia	¿De qué trata?	Ejemplo
Backends ultrarrápidos para JIT	Compilar IR a código máquina en microsegundos, no milisegundos	TPDE (CGO 2026): compila LLVM-IR 8–26x más rápido que LLVM -O0, con rendimiento igual o mejor
Verificación formal en compiladores	Demostrar matemáticamente que las optimizaciones son correctas (no introducen bugs)	NiceToMeetYou (POPL 2026): síntesis automática de abstract transformers verificados para MLIR. Detectó que el 17% de los transformers actuales son más precisos que los que tiene LLVM
Transpilación para aceleradores de IA	Convertir programas científicos (Fortran, Julia, C++) para que corran en TPUs y NPUs de Google	Modelos oceánicos en Julia transpilados a MLIR, ejecutados en miles de GPUs y TPUs distribuidas
Compiladores cuánticos	Traducir circuitos cuánticos (Qiskit, Cirq) a pulsos de hardware real	Qiskit, Cirq, TKET (cada vez más maduros)

5. Importancia en la Ingeniería en Informática actual

Puede que al principio de la carrera muchos estudiantes piensen que los compiladores son un tema teórico, interesante pero alejado de lo que harán en el "mundo real". Nada más lejos de la realidad. Entender cómo funciona un compilador no es solo un requisito para aprobar la materia; es una herramienta que cambia la forma de programar, depurar y diseñar sistemas. A continuación, las razones por las que el conocimiento de compiladores es relevante para un ingeniero en informática.

5.1. Te convierte en un mejor programador (aunque no construyas un compilador)

Cuando entiendes lo que el compilador hace con tu código, empiezas a escribir mejor código. No es magia ni experiencia mística; es conocimiento concreto:

Si sabes que...	Entonces...
El análisis léxico elimina espacios y comentarios	No te obsesiones con la "elegancia" de los espacios en blanco; el compilador no los ve
El análisis sintáctico construye un AST	Puedes imaginar cómo se representa tu código internamente y detectar ambigüedades
Existen optimizaciones como propagación de constantes o eliminación de código muerto	Escribes código claro y legible, y dejas que el compilador haga lo suyo en lugar de micro-optimizar a mano
El backend asigna registros	Entiendes por qué a veces usar menos variables locales puede hacer el código más rápido

En pocas palabras: entender el compilador te quita el miedo y te da control. Sabes qué esperar, qué se puede optimizar y qué no.

5.2. Las técnicas de compiladores están en todas partes

Quizás no lo notes, pero las fases de un compilador (análisis léxico, sintáctico, semántico) aparecen en muchas herramientas cotidianas:

Herramienta / Tecnología	¿Qué técnica de compilador usa?
Linters (ESLint, Clang-Tidy, Pylint)	Análisis sintáctico y semántico (recorren el AST buscando malas prácticas)
Formateadores (Prettier, Black, gofmt)	Análisis sintáctico (parsean el código y lo reescriben con un estilo consistente)
Validadores de JSON/XML/YAML	Análisis léxico y sintáctico (verifican que el archivo esté bien formado)
Buscadores de código (ripgrep, grep avanzado)	Análisis léxico (tokenización) para búsquedas más inteligentes que simple texto
Motores de plantillas (Jinja, ERB, Handlebars)	Análisis sintáctico (parsean el template y lo convierten en código ejecutable)
Infraestructura como Código (IaC) (Terraform, Kubernetes)	Validación sintáctica y semántica de archivos YAML/JSON antes de desplegar

5.3. Es la base para diseñar lenguajes y herramientas propias

En ingeniería en informática, no siempre se usa un lenguaje de propósito general. A veces necesitas crear un lenguaje de dominio específico (DSL) para tu equipo, tu empresa o tus clientes. Ejemplos comunes:

DSL	¿Para qué sirve?
SQL	Consultas a bases de datos
Expresiones regulares	Búsqueda de patrones en texto
HTML/CSS	Maquetación web
Markdown	Formato de texto ligero
GraphQL	Consultas a APIs
Terraform HCL	Definición de infraestructura en la nube

Si entiendes las fases de un compilador, puedes **construir tu propio DSL** cuando lo necesites. No tienes que hacer un compilador completo desde cero; a veces basta con un parser (usando ANTLR, PEG, etc.) que convierta tu lenguaje en algo ejecutable (un script, JSON, consultas a una API, etc.).

5.4. Es clave en seguridad informática (ciberseguridad)

Los **decompiladores** son esenciales en auditorías de seguridad. Permiten:

- ✓ Analizar binarios maliciosos sin tener el código fuente (malware, ransomware)
- ✓ Encontrar vulnerabilidades en software propietario (cuando no tienes acceso al repositorio)
- ✓ Recuperar código fuente perdido (si tienes el ejecutable pero perdiste el original)
- ✓ Validar que los compiladores cruzados y toolchains no introduzcan errores

Herramientas como **Ghidra** (desarrollada por la NSA y liberada al público) o **IDA Pro** son decompiladores de uso profesional en ciberseguridad. Y todas esas herramientas, en el fondo, invierten las fases de un compilador: toman binario, lo desensamblan, infieren

estructuras de datos y generan pseudocódigo de alto nivel. Sin comprensión de cómo funciona un compilador, no se puede construir un decompilador que funcione.

5.5. Abre puertas a áreas especializadas y emergentes

El conocimiento de compiladores no te limita a trabajar en "fábricas de compiladores" (que son muy pocas). Al contrario, te permite acceder a áreas de alta demanda:

Área	Relación con compiladores
Computación de alto rendimiento (HPC)	Optimizar código para supercomputadoras, GPUs y clusters
Compiladores cuánticos	Convertir circuitos cuánticos en pulsos de hardware (Qiskit, Cirq)
IA/ML	Optimizar grafos de tensores con TVM, MLIR o XLA
Blockchain / Web3	Compiladores para contratos inteligentes (Solidity, Move, Rust en blockchains)
Edge computing / IoT	Generar binarios pequeños y eficientes para dispositivos con recursos limitados
Seguridad ofensiva y defensiva	Análisis de binarios, ingeniería inversa, detección de supply chain attacks

5.6. Desarrolla pensamiento abstracto y resolución de problemas

Finalmente, y quizás lo más importante a nivel formativo: construir un compilador (aunque sea pequeño) es uno de los ejercicios de programación más completos que puedes hacer. Te obliga a manejar:

- ✓ Estructuras de datos complejas (grafos, árboles, tablas hash, pilas)
- ✓ Recursión (el AST se recorre recursivamente, los parsers LL son recursivos)
- ✓ Manejo de errores robusto
- ✓ Optimización y compromisos (tiempo de compilación vs. tiempo de ejecución)
- ✓ El clásico problema de "dividir y conquistar" en su máxima expresión

Este tipo de pensamiento se transfiere a cualquier otra área de la informática, incluso si nunca vuelves a tocar un compilador en tu vida profesional.

Conclusión.

A lo largo de este trabajo de investigación, hemos recorrido en detalle las fases que componen un compilador, desde el análisis léxico hasta la generación de código objeto. Lo primero que podemos concluir es que un compilador no es simplemente un "traductor mágico" de texto a binario, sino un sistema formal perfectamente estructurado y comprensible. Su arquitectura se divide en dos grandes etapas bien diferenciadas: el Front-End, encargado de entender el código fuente (analizando su forma, su estructura y su significado), y el Back-End, responsable de generar código ejecutable (creando una representación intermedia, optimizándola y traduciéndola finalmente a código máquina). A estas dos etapas se suman componentes transversales como la tabla de símbolos y el manejo de errores, que acompañan todo el proceso de compilación.

Otra conclusión importante es que, aunque los fundamentos teóricos de los compiladores fueron establecidos hace décadas por autores como Aho, Lam, Sethi y Ullman, el campo está lejos de ser un tema "muerto" o puramente académico. Todo lo contrario. Las tendencias modernas que revisamos, como la compilación Just-In-Time (JIT) que utilizan motores como V8 o la JVM, las arquitecturas modulares como LLVM que reutilizan backends entre lenguajes, y la irrupción de MLIR como infraestructura multi-nivel para dominios específicos, demuestran que los compiladores están en constante evolución. Además, tecnologías como WebAssembly (WASM) han expandido el alcance de la compilación hacia entornos de ejecución seguros y portables, tanto dentro como fuera de los navegadores.

Una de las tendencias más innovadoras y quizás la más disruptiva es la aplicación de inteligencia artificial al proceso de compilación. Como vimos en la literatura más reciente (CGO 2026, POPL 2026), ya se están utilizando agentes de aprendizaje por refuerzo para decidir qué optimizaciones aplicar en MLIR, logrando políticas que superan a las heurísticas escritas a mano. También se están diseñando sistemas compilador-runtime co-diseñados con modelos de ML ligeros para elegir en tiempo real la mejor variante de ejecución según las condiciones del hardware. Esto sugiere que, en un futuro cercano, la optimización de código será una tarea compartida entre ingenieros humanos y agentes de IA.

Desde el punto de vista formativo, el estudio de los compiladores tiene un valor que trasciende la mera aprobación de una asignatura. Entender cómo funciona un compilador nos convierte en mejores programadores: sabemos qué esperar de la herramienta, entendemos por qué ciertos patrones de código son más eficientes que otros, y podemos leer e interpretar los mensajes de error con mayor claridad. Además, las técnicas de análisis léxico, sintáctico y semántico aparecen en múltiples herramientas cotidianas como linters, formateadores, validadores de JSON o lenguajes de dominio específico (DSL). Un ingeniero en informática que domina estos conceptos tiene una ventaja competitiva considerable, pues puede diseñar

sus propias herramientas, auditar código binario en tareas de ciberseguridad, o incluso contribuir a compiladores de lenguajes emergentes.

Por último, queremos destacar la relevancia de este conocimiento para nuestra carrera en la UNEG. La Ingeniería en Informática no se limita a usar lenguajes de programación, sino a comprender cómo funcionan por dentro, cuáles son sus límites y cómo se pueden extender. El compilador es, en ese sentido, una de las herramientas más fundamentales de nuestra disciplina. Como dijo el profesor Félix Márquez en el material del curso: "La programación es un hermoso proceso cognitivo que no es solo decirle a una máquina qué hacer, es aprender a comunicarte con claridad en un universo que no perdona errores. El compilador, en ese sentido, es el primer maestro formal de esa disciplina". Nosotros, como grupo "Los ASTRonautas", hemos intentado explorar ese universo de los lenguajes, un nodo a la vez, y esperamos que este informe refleje el esfuerzo y la comprensión alcanzada.

Referencias.

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compiladores: Principios, Técnicas y Herramientas* (2da ed.). Addison-Wesley.
- Louden, K. C. (2004). *Construcción de Compiladores: Principios y Práctica*. Editorial Thompson.
- La Evolución y el Futuro de los Compiladores en la Era Digital. (2024, 20 de abril). *Puerto 53*. Recuperado de <https://puerto53.wordpress.com/2024/04/20/la-evolucion-y-el-futuro-de-los-compiladores-en-la-era-digital/>
- Historia de la construcción de los compiladores. (s.f.). En *Wikipedia*. Recuperado el 9 de mayo de 2026 de https://es.wikipedia.org/wiki/Historia_de_la_construccion_de_los_compiladores
- Historia de los compiladores. (s.f.). *Timetoast*. Recuperado de <https://www.timetoast.com/timelines/historia-de-los-compiladores-ee0d2574-bfe5-47f6-9c31-c5787390d642>
- Historia de los compiladores | FORTRAN-PASCAL-C. (s.f.). *SlideShare*. Recuperado de <https://es.slideshare.net/slideshow/historia-de-los-compiladores/95993408>
- ¿Qué fue primero el programa o el compilador? (s.f.). *Tríbalyte Technologies*. Recuperado de <https://trbl-services.eu/blog-programa-o-compilador/>
- Hernandez, M. (s.f.). *Analizador semántico*. *Compiladores*. Recuperado de <https://compiladores57.webnode.mx/analizador-semantico/>
- *Analizador sintáctico*. (s.f.). *Compiladores*. Recuperado de <https://compiladores57.webnode.mx/analizador-sintactico/>
- Rodríguez, C. (s.f.). *Expresiones Regulares en Flex*. *Apuntes de Lenguajes y Compiladores - ULL*. Recuperado de <https://crguezl.github.io/ull-etsii-grado-pl-apuntes/node82.html>
- *Análisis Léxico - Compiladores*. (s.f.). *CC4 GitBook*. Recuperado de <https://cc4.gitbook.io/cc4/proyectos/untitled>

- Tito, C. (2019, 9 de febrero). *Crea tu propio compilador – Cap. 7 – Análisis sintáctico*. Blog de Tito. Recuperado de <https://blogdetito.com/2019/02/09/crea-tu-propio-compilador-parte-7/>
- ¿Cuál es el propósito de una tabla de símbolos en un compilador? (s.f.). LinkedIn Learning. Recuperado de <https://www.linkedin.com/advice/1/what-purpose-symbol-table-compiler-skills-computer-science-8csde>
- Fgg LinkedIn Learning. (s.f.). *¿Cuál es el propósito de una tabla de símbolos en un compilador?* Recuperado de <https://www.linkedin.com/advice/1/what-purpose-symbol-table-compiler-skills-computer-science-8csde>
- SecondState. (2026, 4 de marzo). **WasmEdge Community Update: GSoC 2026 Proposals and AI-Driven Sandboxing**. <https://www.secondstate.io/articles/wasmedge-community-update-mar-26/>
- Kaiser, H. (2026, 5 de mayo). *Co-Designing Integrated AI-Enabled Compiler-Runtime Systems for Future HPC Architectures*. C++Now 2026. <https://schedule.cppnow.org/session/2026/co-designing-integrated-ai-enabled-compiler-runtime-systems-for-future-hpc-architectures/>